NURa Hand-in 2

Silvan Toet (S1827332)

Date

Write a short description of your solution in each section. Explain your choices. Explicitly draw conclusions from your results.

You can refer to selected lines of code where you think it is necessary. However, you must provide the full code relevant to each (sub)question below it.

Don't forget to describe your figures and mention what conclusions you draw from them!

1 Satellite galaxies around a massive central

1.1 (1a) Determining the normalization

The spherical volume element dV can be defined as follows: We know that the volume can be computed in spherical coordinates using the triple integral

$$\iiint_V n(x)dV = \int_{\phi=0}^{2\pi}\int_{\theta=0}^{\pi}\int_{r=0}^{r_{max}} n(x)r^2 \sin\theta \, dr \, d\theta \, d\phi \tag{1}$$

$$= 4\pi A\langle N_{sat}\rangle \int_{x=0}^{x_{max}=5} x^2 \left(\frac{x}{b}\right)^{a-3} \exp(-\left(\frac{x}{b}\right)^c)dx = \langle N_{sat}\rangle \tag{2}$$

$$-> A = \frac{1}{4\pi \int_{x=0}^{x_{max}=5} x^2 \left(\frac{x}{b}\right)^{a-3} \exp(-\left(\frac{x}{b}\right)^c)dx} \tag{3}$$

Since n(x) where x is the radius relative to virial radius $x = r/r_{vir}$ is only dependent on r, we can easily solve for θ and ϕ. Dividing by $\langle N_{sat}\rangle$ on both sides and rearranging gives us an expression for A.

Next we use Romberg integration (i.e. Richardson extrapolation applied to integration) to numerically integrate the integral in the expression. Romberg integration first applies the extended trapezoid rule for N intervals, then 2N, 4N, ... up to $2^m\times$ N intervals, where m is the order of the initial approximations. Next we use the fact that each successive order's leading error term is exactly a 1/4 times the leading error term of the previous order. Subtracting the two approximations from each other in such a way as to cancel the leading error term results in m-1 new approximations. We can do this subtraction trick m-1 times. Since after each subtraction we no longer need the two values used for this, we can store the initial approximations and all subsequent improvements in a vector of length m. At the end of the algorithm the first element of the vector contains the best estimate of the integral and the best estimate for the error is the absolute difference between the first two elements. Table 1 shows the value of the normalization constant A to full precision.

Table 1: Normalization constant of the satellite galaxy profile.

Normalization constant A
9.19836296311

1.2 Code for the integrator

```python
##### Integrator block #####

import numpy as np


def trapezoid(a, b, func, N: int):
    """Use the extended trapezoid rule to calculate the integral.
    Parameters a and b are start and stop x values of the range to be integrated respectively.
    func is the function to be evaluated.
    N is the number of evaluations.
    """
    xdata = np.linspace(a, b, num=N)
    h = (b - a) / N  # step size
    return h * (0.5 * (func(b) + func(a)) + np.sum(func(xdata[1:-1])))


def romberg_vector_version(a, b, func, N_start: int, order: int = 5, return_error: bool = False):
    """See slide 14 of Lecture 4 annotated slides for the formula used.
    N_start is the initial number of function evaluations. This will double with every order.
    order is the number of initial approximations.
    We return the best estimate for the integral stored at r_0
    If return_error is True, the error estimate abs(r_0 - r_1) is also returned
    """
    romberg_vector = np.zeros(shape=order)
    for j in range(order):
        for i in range(order - j):
            if j == 0:  # the first time we fill the column, we use the trapezoid rule with doubling step sizes per row entry
                N = (2**i) * N_start  # number of intervals doubles with each depth
                romberg_vector[i] = trapezoid(a, b, func, N)
            else:  # Subsequent times we use the update rule
                romberg_vector[i] = (4**j * romberg_vector[i + 1] - romberg_vector[i]) / (4**j - 1)  # General version of 4/3 S_1 - 1/3 S_0
    if return_error:
        return romberg_vector[0], abs(romberg_vector[0] - romberg_vector[1])  # (value, error)
    return romberg_vector[0]
```

1.3 (1b) Sampling the satellite distribution

Describe the sampling method used (e.g. rejection sampling). Explain how the random number generator was implemented and how its randomness was tested. We use rejection sampling to generate 10000 satellite galaxy positions. In rejection sampling we first scale the probability distribution function max value to 1. Then we generate two random numbers:

- one random number between the x ranges of our pdf. In our case this is between x_{min}=0 and x_{max}=5

- another random number between 0 and 1. This number represents the probability that the number is accepted

if y < p(x), we accept the generated x value in our sample. Since y is more likely to lie under p(x) if p(x) is large for that value of x, this incorporates the difference between probabilities for different x values. Hence the statement "2 random numbers in, <1 random number out", because we need two random numbers to generate at most 1 (we might also reject it).

We implement our random number generator as a combination of an 64-bit XOR shift method and a (Multiple) Linear Congruential Generator. Both sub RNGs receive their own starting seed, and base their subsequent generated number on only (a part of) their own previous output. The two random numbers generated by the subgenerators are then added element-wise in the combination-rng.

We could not get BigCrush our DieHard to work to thoroughly test our RNG, but we did plot a histogram of 100000 uniformly generated test values below:

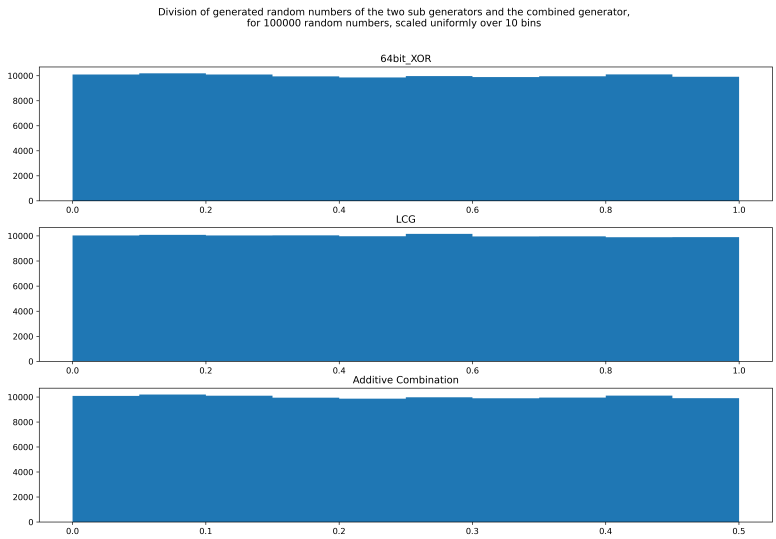

Figure 1: Comparison between the random numbers generated by the 64-bit XOR shift, MLCG and combinatory generator, for 100000 samples spread uniformly in the range (0,1) over 10 bins.

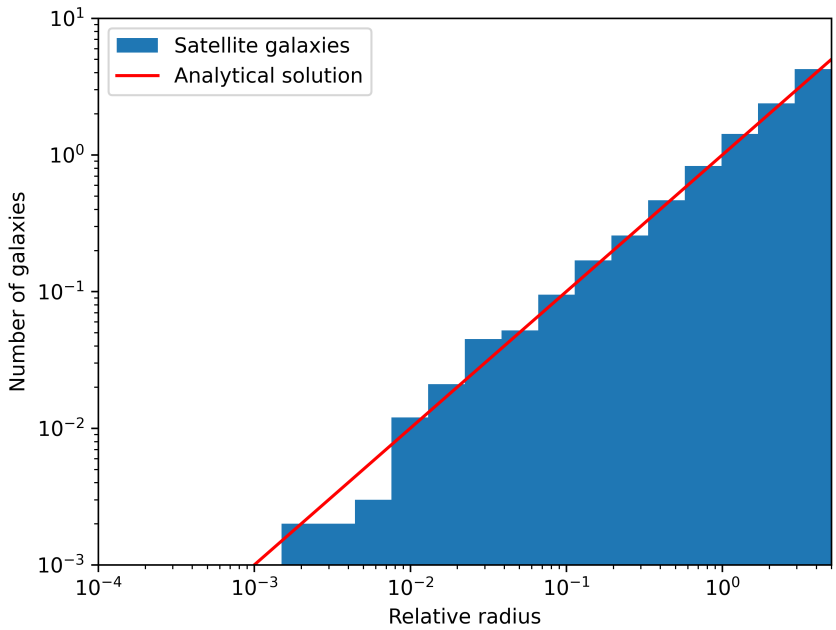

Figure 2: Comparison between the analytical satellite distribution $N(x)$ and the histogram of 10 000 sampled satellite galaxies in logarithmic bins.

Does your sampled distribution agree with the analytical prediction?

1.4 Code for sampling

```
#### Sampler block including RNGs ####

import numpy as np


def rng_64bit_xor_shift(x: np.uint64 = np.uint64(123456789), a_iter: Iterable[np.uint64] = (np.uint64(21), np.uint64(35), np.uint64(4)), size: int = 1, scale_uniform: bool = False):
    """Use the 64-bit XOR-shift method for making a Random Number Generator on slide 14 of lecture 5.
    x is the seed, which should be an unsigned 64 bit integer, not 0.
    We use a numpy variable, as Python built-in variables will automatically extend if bits are moved out of range (which is what we want to use here: moving bits out of range hides info for the user).
    a is an iterable of ints containing the coefficients that will be used to bitshift x by a certain amount before doing an XOR operation with itself.
    Size is the number of random numbers generated. scale_uniform scales the output to the range [0,1]
    """
    x = np.uint64(x)
    a_list = [np.uint64(a) for a in a_iter]  # Convert all coefficients to np.uint64 variables once, to ensure no error occurs if the user does not supply these in np.uint64 format
    if x == 0:
```

```python
        raise ValueError("x cannot have 0 as a starting value")

    random_values_array = np.zeros(size, dtype=np.uint64)
    for i in range(len(random_values_array)):
        for j, a in enumerate(a_list):
            x = x ^ (x >> a) if j % 2 == 0 else x ^ (x << a)  # Even coefficients are used to shift x a bits to the right, odd coefficients are used to shift x a bits to the left
            # This is necessary to ensure randomness: otherwise we would only be changing e.g. the lower significant bits, and numbers would oscillate around the starting seed value
        random_values_array[i] = x
    if scale_uniform:
        return random_values_array / np.iinfo(np.int64).max  # Division operator converts dtypes to np.float64, which is what we want here
    return random_values_array


def lcg(x: np.uint64 = np.uint64(123456789), a: np.uint64 = np.uint64(7654321), c: np.uint64 = np.uint64(3333333), m: np.uint64 = np.uint64(1234543212345), size: int = 1, scale_uniform: bool = False):
    """Implement a (Multiple) Linear Congruential Generator.
    When c=0 we are dealing with a MLCG.
    This RNG is bad on its own but a good building block for more complex RNGs.
    Size is the number of random numbers generated. scale_uniform scales the output to the range [0,1]
    """
    x, a, c, m = np.uint64(x), np.uint64(a), np.uint64(c), np.uint64(m)  # Cast to unsigned integers should user forget to do so
    random_values_array = np.zeros(size, dtype=np.uint64)
    for i in range(len(random_values_array)):
        x = (a * x + c) % m
        random_values_array[i] = x
    if scale_uniform:
        return random_values_array / np.iinfo(np.int64).max
    return random_values_array


def additive_combined_rng(
    x1: np.uint64 = np.uint64(123456789),
    x2: np.uint64 = np.uint64(987654321),
    a_iter: Iterable[np.uint64] = (np.uint64(21), np.uint64(35), np.uint64(4)),
    a: np.uint64 = np.uint64(7654321),
    c: np.uint64 = np.uint64(3333333),
    m: np.uint64 = np.uint64(1234543212345),
    size: int = 1,
    scale_uniform: bool = True,
):
    P_64 = rng_64bit_xor_shift(x=x1, a_iter=a_iter, size=size, scale_uniform=scale_uniform)
    P_lcg = lcg(x=x2, a=a, c=c, m=m, size=size, scale_uniform=scale_uniform)
    P_add = P_64 + P_lcg
    if scale_uniform:
        return P_add / 2  # Both sub generator outputs have been scaled to [0,1) range, so here we need to divide by 2
    return P_add


def sampler(
    dist: callable,
    min: float,
    max: float,
```

```
    Nsamples: int,
    args: tuple = (),
) -> np.ndarray:
    """
    Sample a distribution using sampling method of your choice

    Parameters
    dist : callable
        Distribution to sample
    min :
        Minimum value for sampling
    max : float
        Maximum value for sampling
    Nsamples : int
        Number of samples
    args : tuple, optional
        Arguments of the distribution to sample, passed as args to dist

    Returns
    -------
    sample: ndarray
        Values sampled from dist, shape (Nsamples,)
    """

    # Scale p(x) top to 1. So basically divide n_x (or p_x??) by its maximum value
    # Generate a random number from the distribution that we want a value from, so n(x) in our case
    # Then generate a separate random number from a uniform distribution [0,1), which we interpret as the probability
    # Then we accept x into our sample if y < p(x): the higher p(x) is for that value of x, the more likely x should be, and therefore the more likely that y is smaller than p(x)
    # First

    P_uniform = additive_combined_rng(size=10000)
    P_ab = a + (b - a) * P_uniform
    n_x_scaled = n_x / n_x_max

    return np.zeros(Nsamples)
```

1.5 (1c) Selecting and sorting satellites

Describe the selection of 100 galaxies (equal probability, no duplicates, no rejection). Explain the sorting algorithm you used.

We are asked to select 100 galaxies with equal probability, without duplicates and without rejection of a generated sample. As explained during the lecture, a Linear Congruential Generator is not a great RNG on its own, partially due to the fact it cannot generate duplicate samples before it starts to repeat itself. Once it starts to repeat itself, the entire sequence repeats (becomes periodic). But this means that if we choose our LCG parameters (a,c and m) right, we can ensure that we get 100 random numbers with equal probability without a duplicate! A quick search on LCGs on wikipedia (`https://en.wikipedia.org/wiki/Linear_congruential_generator`) reveals that if we satisfy the Hull-Dobell theorem, we are guaranteed to get a full sequence of period m for every seed value x. The three criteria to be met are:

1. m and c are comprime
2. a - 1 is divisible by all prime factors of m

3. a - 1 is divisible by 4 if m is divisible by 4

We choose m to be the size of the array that we sample from, in our case 10000 which is the number of initally generated satellite galaxies. Next we choose a and c to satisfy the above criteria. m has as prime factors 2 and 5. So choose c as odd and not divisible by 5. a must be divisible by 2, 4 and 5. We choose a=21 and c=37. The generated indices binned are shown in figure 3.

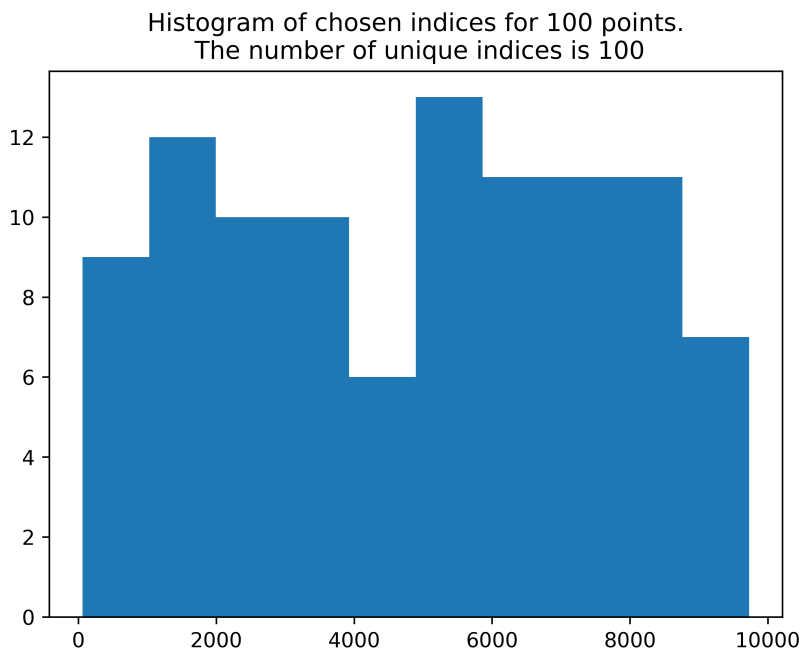

Figure 3: Generated indices. The title figure contains a check to see if there are indeed 100 unique indices.

Now we sort the 100 selected galaxies using our sorting algorithm. Our first attempt was to use our quicksort algorithm. The quicksort algorithm avoids the pitfall of simple sorting techniques such as selection sort that need N^2 comparisons for an array of length N. It does this by splitting the array into smaller and smaller subarrays and sorting these recursively. We first use selection sort to choose the pivot element from the first, middle and last element. Then we start swapping elements to the left of the pivot that are larger than the pivot with elements to the right of the pivot that are smaller than the pivot. This ensures that after 1 recursion, we end up with two subarrays that, although they are still unsorted themselves, their elements are in the correct position compared to the pivot element. Doing this recursively eventually sorts the entire array with an order O(N log N) comparisons.

Unfortunately, checking the array after the quicksort call revealed there were some values not sorted correctly. We therefore resort to our own selection sort as a fallback option for this assignment. We have left both algorithms in the code for inspection and will be very happy if any of the graders can spot the mistake in the quicksort algorithm.

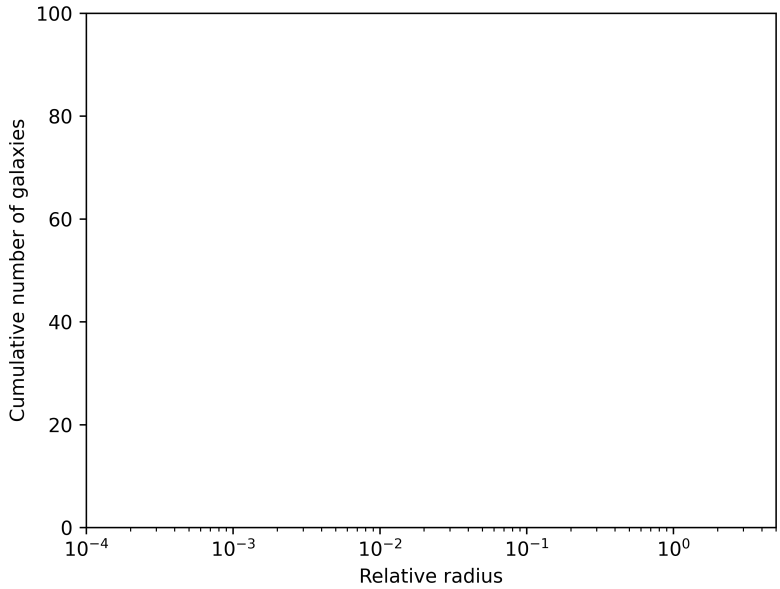

Figure 4: Cumulative number of selected galaxies as a function of relative radius.

1.6 Code for selection and sorting

```
#### Sorting and Selection block ####


import numpy as np


def sort_array(
    arr: np.ndarray,
    inplace: bool = False,
) -> np.ndarray:
    """Sort a 1D array using a sorting algorithm of your choice

    Parameters
    ----------
    arr : ndarray
        Input array to be sorted
    inplace : bool, optional
        If True, sort the array in-place
        If False, return a sorted copy

    Returns
    -------
    sorted_arr : ndarray
        Sorted array (same shape as arr)

    """
    if inplace:
        sorted_arr = arr
    else:
```

```
        sorted_arr = arr.copy()

    # TODO: sort sorted_arr in-place here

    return sorted_arr


def choice(arr: np.ndarray, size: int = 1) -> np.ndarray:
    """Choose given number of random elements from an array, without replacement

    Parameters
    ----------
    arr : ndarray
        Array to shuffle
    size : int, optional
        Number of elements to pick from array
        The default is 1

    Returns
    -------
    chosen : ndarray
        Randomly chosen elements from arr, shape (size,)
    """
    # TODO: Implement your choice function here, e.g. by using Fisher-Yates shuffling
    return arr[:size].copy()
```

1.7 (1d) Numerical derivative

Table 2: Derivative of the satellite number density at $x = 1$.

Method	Value
Analytical	0
Numerical	0

Do your analytical and numerical derivatives agree? Why?

1.8 Code for derivative calculation

```
##### Derivative block #####


import numpy as np


def dn_dx(x: float | np.ndarray, A: float, Nsat: float, a: float, b: float, c: float) -> float | np.ndarray:
    """Analytical derivative of number density provide

    Parameters
    ----------
    x : ndarray
        Radius in units of virial radius; x = r / r_virial
    A : float
        Normalisation
    Nsat : float
        Average number of satellites
    a : float
```

```
        Small-scale slope
    b : float
        Transition scale
    c : float
        Steepness of exponential drop-off

    Returns
    -------
    float | ndarray
        Same type and shape as x. Derivative of number density of
        satellite galaxies at given radius x.
    """
    # TODO: Write the analytical derivative of n(x) here
    return 0.0


def finite_difference(function: callable, x: float | np.ndarray, h: float) -> float | np.ndarray:
    """
    A building block to compute derivative using finite differences

    Parameters
    ----------
    function : callable
        Function to differentiate
    x : float | ndarray
        Value(s) to evaluate derivative at
    h : float
        Step size for finite difference

    Returns
    -------
    dy : float | ndarray
        Derivative at x
    """
    # TODO: Implement finite difference method
    return 0.0


def compute_derivative(
    function: callable,
    x: float | np.ndarray,
    h_init: float,
    # For Ridders use parameters below:
    # d: float, # Factor by which to decrease h_init every iteration
    # eps: float, # Relative error
    # max_iters: int = 10, 3 Maximum number of iterations before exiting
) -> float | np.ndarray:
    """
    Function to compute derivative

    Parameters
    ----------
    function : callable
        Function to differentiate
    x : float | ndarray
        Value(s) to evaluate derivative at
    h_init : float
        Initial step size for finite difference

    Returns
    -------
```

```
    df : float | ndarray
        Derivative at x
    """
    # TODO: Implement derivative
    return 0.0
```

2 Heating and cooling in HII regions

2.1 (2a) Photoionization heating and radiative recombination cooling

Describe the implementation of your root-finding method (e.g. bisection, Newton-Raphson).

Table 3: Equilibrium temperature for the simplified heating–cooling model.

Temperature [K]	Absolute error	Relative error
0	0.000e+00	0.000e+00

How many iterations it took? Discuss the convergence of your root finder.

2.2 (2b) More realistic model

Table 4: Equilibrium temperature for different gas densities.

Density n_H [cm^{-3}]	Equilibrium temperature [K]
10^{-4}	0
1	0
10^4	0

Discuss the results. What is the difference for the low, intermediate and high gas density?

2.3 Code for Exercise 2

```
import numpy as np

# Constants (mind the units!)

psi = 0.929
Tc = 1e4 # K
Z = 0.015
k = 1.38e-16  # erg/K
aB = 2e-13  # cm^3 / s
A = 5e-10
xi = 1e-15


# There's no need for nH nor ne as they cancel out
def equilibrium1(T, Z, Tc, psi):
    return psi * Tc * k - (0.684 - 0.0416 * np.log(T / (1e4 * Z * Z))) * T * k


def equilibrium2(T, Z, Tc, psi, nH, A, xi, aB):
    return (
        (
```

```python
            psi * Tc
            - (0.684 - 0.0416 * np.log(T / (1e4 * Z * Z))) * T
            - 0.54 * (T / 1e4) ** 0.37 * T
        )
        * k
        * nH
        * aB
        + A * xi
        + 8.9e-26 * (T / 1e4)
    )


# Derivative function, might be useful if using Newton-Raphson method for root finding
# def equilibrium2_deriv(T, nH):
#     # TODO: Compute derivative of equilibrium2 with respect to T
#     return 0.0


#### root finder ####


def root_finder(
    func: callable,  # add derivative if using Newton-Raphson
    bracket: tuple,
    atol: float = 1e-6,
    rtol: float = 1e-6,
    max_iters: int = 100,
) -> tuple[float, float, float]:
    """
    Find a root of a function

    Parameters
    ----------
    func : callable
        Function to find root of
    bracket : tuple
        Bracket for which to find first secant
    atol : float, optional
        Absolute tolerance.
        The default is 1e-6
    rtol : float, optional
        Relative tolerance.
        The default is 1e-6
    max_iters: int, optional
        Maximum number of iterations.
        Teh default is 100

    Returns
    -------
    root : float
        Approximate root
    aerr : float
        Absolute error
    rerr : float
        Relative error
    """
    # TODO: Implement root finder (e.g. bisection, false-position, Newton-Raphson)
    return 0.0, 0.0, 0.0


def main():
```

```python

    # Initial bracket
    bracket = (1, 1e7)

    root, aerr, rerr = 0.0, 0.0, 0.0  # replace with your root finder

    with open("Calculations/equilibrium_temp_simple.txt", "w") as f:
        f.write(f"{root:.12g} & {aerr:.3e} & {rerr:.3e}")
    #### 2b ####

    # Initial bracket
    bracket = (1, 1e15)

    for nH in [1e-4, 1, 1e4]:

        root, aerr, rerr = 0.0, 0.0, 0.0  # replace with your root finder
        if nH == 1e-4:
            with open("Calculations/equilibrium_low_density.txt", "w") as f:
                f.write(f"{root:.12g}")
        elif nH == 1:
            with open("Calculations/equilibrium_mid_density.txt", "w") as f:
                f.write(f"{root:.12g}")
        elif nH == 1e4:
            with open("Calculations/equilibrium_high_density.txt", "w") as f:
                f.write(f"{root:.12g}")


if __name__ == "__main__":
    main()
```

3 Conclusions